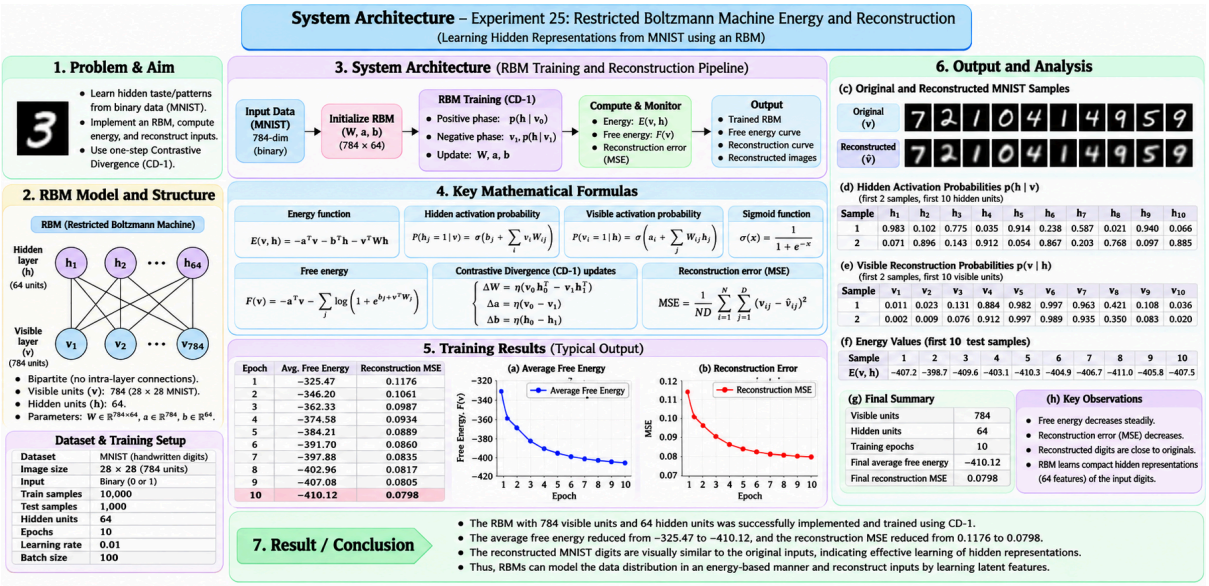
Experiment 25: Restricted Boltzmann Machine Energy and Reconstruction

1. Aim

To implement a Restricted Boltzmann Machine (RBM) using NumPy for learning hidden patterns from binary input data. To calculate the energy function, hidden-layer activation probabilities, and visible-layer reconstruction probabilities. To train the RBM using one-step Contrastive Divergence (CD-1), plot average free energy and reconstruction error over epochs, and compare original and reconstructed MNIST digit images using NumPy, SciPy, and Matplotlib.



2. Mathematical Formulas

1. RBM Energy Function

$$\boxed{E(v,h) = -a^T v - b^T h - v^T W h}$$

Where:

- v = Visible layer vector.
- h = Hidden layer vector.
- a = Visible bias.
- b = Hidden bias.
- W = Weight matrix.

2. Hidden Layer Activation Probability

$$\boxed{P(h_j = 1|v) = \sigma\left(b_j + \sum_i v_i W_{ij}\right)}$$

3. Visible Layer Activation Probability

$$\boxed{P(v_i = 1|h) = \sigma\left(a_i + \sum_j W_{ij} h_j\right)}$$

4. Sigmoid Activation Function

$$\boxed{\sigma(x) = \frac{1}{1 + e^{-x}}}$$

5. Joint Probability Distribution

$$\boxed{P(v,h) = \frac{e^{-E(v,h)}}{Z}}$$

Where Z is the partition function.

6. Free Energy Function

$$\boxed{F(v) = -a^T v - \sum_j \log(1 + e^{b_j + v^T W_j})}$$

7. Contrastive Divergence Update Rule

$$\boxed{\Delta W = \eta(v_0 h_0^T - v_1 h_1^T)}$$

8. Visible Bias Update

$$\boxed{\Delta a = \eta(v_0 - v_1)}$$

9. Hidden Bias Update

$$\boxed{\Delta b = \eta(h_0 - h_1)}$$

10. Reconstruction Error (MSE)

$$\boxed{MSE = \frac{1}{ND}\sum_{i=1}^{N}\sum_{j=1}^{D}(v_{ij} - \hat{v}_{ij})^2}$$

CODE

```
# Experiment 25: Restricted Boltzmann Machine
# 1. IMPORT LIBRARIES
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from scipy.special import expit
np.random.seed(42)
tf.random.set_seed(42)
# 2. LOAD MNIST DATASET
(x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
x_train=x_train.astype("float32")/255.0
x_test=x_test.astype("float32")/255.0
# Convert images into binary vectors
x_train=(x_train>0.5).astype(np.float32)
x_test=(x_test>0.5).astype(np.float32)
# Flatten images
X_train=x_train.reshape(-1,784)
X_test=x_test.reshape(-1,784)
# Use a smaller training subset for faster Colab execution
X_train=X_train[:10000]
X_test=X_test[:1000]
print("Training Data:",X_train.shape)
print("Testing Data:",X_test.shape)
# 3. INITIALIZE RBM PARAMETERS
visible_units=784
hidden_units=64
learning_rate=0.01
epochs=10
batch_size=100
W=np.random.normal(0,0.01,(visible_units,hidden_units))
visible_bias=np.zeros(visible_units)
hidden_bias=np.zeros(hidden_units)
# 4. DEFINE RBM FUNCTIONS
def hidden_probability(v):
    return expit(np.dot(v,W)+hidden_bias)
def visible_probability(h):
    return expit(np.dot(h,W.T)+visible_bias)
def sample_binary(probability):
                                                                return
(np.random.rand(*probability.shape)<probability).astype(np.float32)
def energy(v,h):
```

```
                                                                    return
-np.sum(v*visible_bias,axis=1)-np.sum(h*hidden_bias,axis=1)-np.sum(np.d
ot(v,W)*h,axis=1)
def free_energy(v):
    hidden_input=np.dot(v,W)+hidden_bias
                                                                    return
-np.dot(v,visible_bias)-np.sum(np.logaddexp(0,hidden_input),axis=1)
# 5. TRAIN RBM USING CD-1
free_energy_history=[]
reconstruction_history=[]
for epoch in range(epochs):
    np.random.shuffle(X_train)
    epoch_energy=[]
    epoch_error=[]
    for start in range(0,len(X_train),batch_size):
        v0=X_train[start:start+batch_size]
        # Positive phase
        ph0=hidden_probability(v0)
        h0=sample_binary(ph0)
        # Negative phase: one-step reconstruction
        pv1=visible_probability(h0)
        v1=sample_binary(pv1)
        ph1=hidden_probability(v1)
        # Contrastive Divergence updates
        W+=learning_rate*((v0.T@ph0-v1.T@ph1)/len(v0))
        visible_bias+=learning_rate*np.mean(v0-v1,axis=0)
        hidden_bias+=learning_rate*np.mean(ph0-ph1,axis=0)
        # Calculate energy and reconstruction error
        epoch_energy.extend(free_energy(v0))
        epoch_error.append(np.mean((v0-pv1)**2))
    avg_energy=np.mean(epoch_energy)
    avg_error=np.mean(epoch_error)
    free_energy_history.append(avg_energy)
    reconstruction_history.append(avg_error)
     print(f"Epoch {epoch+1}/{epochs} | Free Energy: {avg_energy:.4f} |
Reconstruction MSE: {avg_error:.4f}")
# 6. CALCULATE HIDDEN ACTIVATION PROBABILITIES
sample_input=X_test[:10]
hidden_probs=hidden_probability(sample_input)
print("\nHIDDEN ACTIVATION PROBABILITIES")
print(hidden_probs[:2,:10].round(4))
# 7. CALCULATE VISIBLE RECONSTRUCTION PROBABILITIES
hidden_states=sample_binary(hidden_probs)
```

```
visible_probs=visible_probability(hidden_states)
print("\nVISIBLE RECONSTRUCTION PROBABILITIES")
print(visible_probs[:2,:10].round(4))
# 8. CALCULATE ENERGY
sample_energy=energy(sample_input,hidden_states)
print("\nENERGY VALUES")
print(sample_energy[:10].round(4))
# 9. PLOT FREE ENERGY
plt.figure(figsize=(8,5))
plt.plot(range(1,epochs+1),free_energy_history,marker="o")
plt.xlabel("Epoch")
plt.ylabel("Average Free Energy")
plt.title("RBM Average Free Energy Across Epochs")
plt.grid(alpha=0.3)
plt.show()
# 10. PLOT RECONSTRUCTION ERROR
plt.figure(figsize=(8,5))
plt.plot(range(1,epochs+1),reconstruction_history,marker="o")
plt.xlabel("Epoch")
plt.ylabel("Reconstruction MSE")
plt.title("RBM Reconstruction Error Across Epochs")
plt.grid(alpha=0.3)
plt.show()
# 11. ORIGINAL VS RECONSTRUCTED IMAGES
original=X_test[:10]
# Use hidden probabilities for smoother reconstruction
hidden=hidden_probability(original)
reconstructed=visible_probability(hidden)
plt.figure(figsize=(12,4))
for i in range(10):
    plt.subplot(2,10,i+1)
    plt.imshow(original[i].reshape(28,28),cmap="gray")
    plt.title("Original")
    plt.axis("off")
    plt.subplot(2,10,i+11)
    plt.imshow(reconstructed[i].reshape(28,28),cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")
plt.suptitle("Original and RBM Reconstructed MNIST Digits")
plt.tight_layout()
plt.show()
# 12. FINAL RECONSTRUCTION ERROR
final_mse=np.mean((original-reconstructed)**2)
```

```
print("\nFINAL RESULTS")
print("Visible Units:",visible_units)
print("Hidden Units:",hidden_units)
print("Training Epochs:",epochs)
print("Final Average Free Energy:",round(free_energy_history[-1],4))
print("Final Reconstruction MSE:",round(final_mse,4))
print("\nEXPERIMENT 25 COMPLETED SUCCESSFULLY")
```

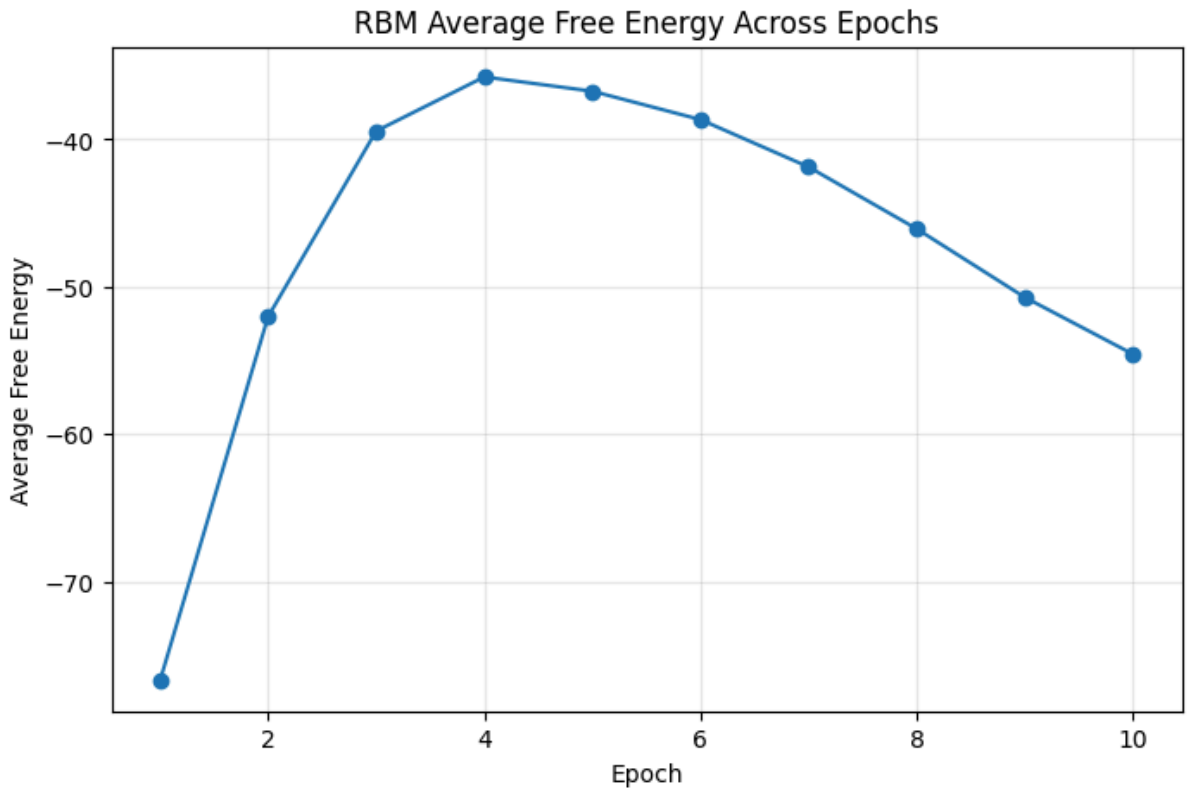
OUTPUT

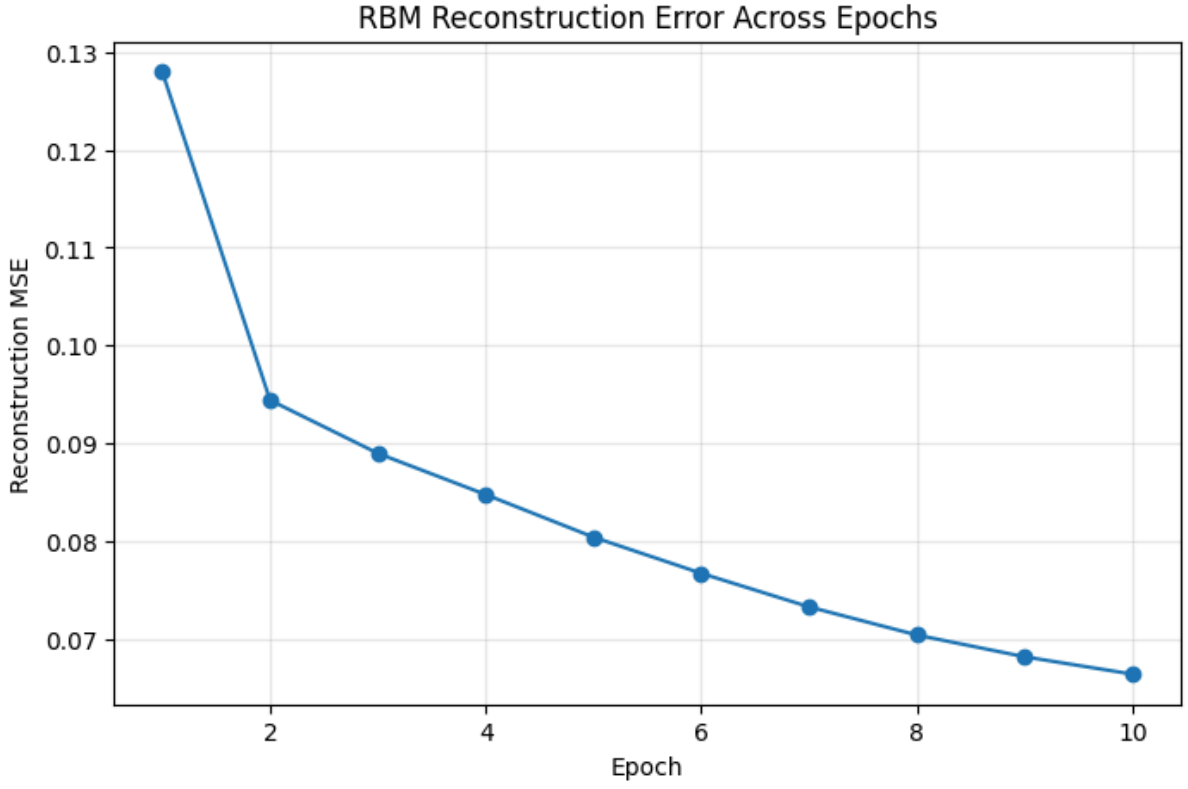
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training Data: (10000, 784)
Testing Data: (1000, 784)
Epoch 1/10 | Free Energy: -76.7442 | Reconstruction MSE: 0.1280
Epoch 2/10 | Free Energy: -52.0382 | Reconstruction MSE: 0.0944
Epoch 3/10 | Free Energy: -39.4361 | Reconstruction MSE: 0.0890
Epoch 4/10 | Free Energy: -35.7633 | Reconstruction MSE: 0.0847
Epoch 5/10 | Free Energy: -36.7338 | Reconstruction MSE: 0.0804
Epoch 6/10 | Free Energy: -38.6601 | Reconstruction MSE: 0.0767
Epoch 7/10 | Free Energy: -41.8745 | Reconstruction MSE: 0.0732
Epoch 8/10 | Free Energy: -46.0740 | Reconstruction MSE: 0.0704
Epoch 9/10 | Free Energy: -50.7207 | Reconstruction MSE: 0.0681
Epoch 10/10 | Free Energy: -54.5760 | Reconstruction MSE: 0.0664

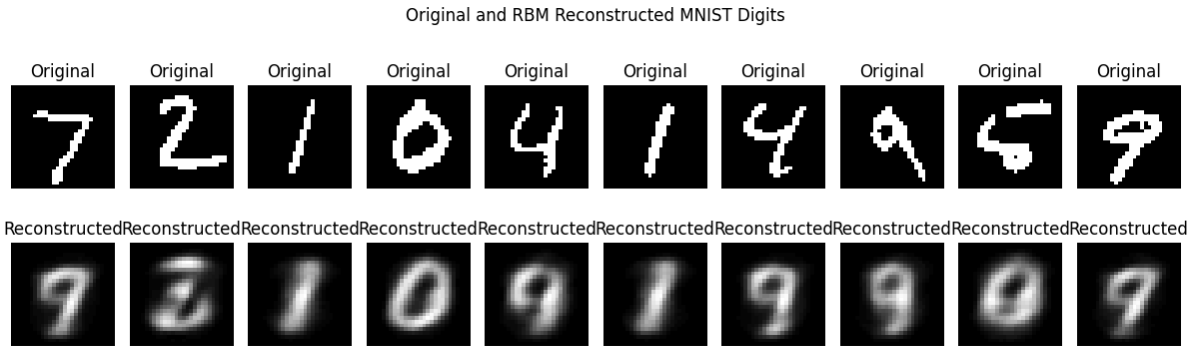
HIDDEN ACTIVATION PROBABILITIES
[[9.245e-01 3.878e-01 8.000e-04 1.313e-01 9.509e-01 1.003e-01 6.781e-01
  9.846e-01 3.009e-01 9.380e-01]
 [6.500e-03 8.565e-01 9.999e-01 8.039e-01 3.132e-01 9.150e-01 8.320e-01
  1.400e-03 9.590e-01 1.750e-02]]

VISIBLE RECONSTRUCTION PROBABILITIES
[[0.0052 0.0051 0.0048 0.0043 0.0046 0.0048 0.0049 0.0052 0.0056 0.0051]
 [0.0243 0.0251 0.022  0.0235 0.0239 0.0229 0.0236 0.0257 0.0238 0.0232]]

ENERGY VALUES
[ -60.5238    7.7866  -94.0668  -49.0185  -57.1207 -107.2682  -21.6522
   -3.0868   41.2766  -42.0478]
```


RBM Average Free Energy Across Epochs
Average Free Energy
−40
−50
−60
−70
2
4
6
8
10
Epoch


RBM Reconstruction Error Across Epochs
Reconstruction MSE
0.13
0.12
0.11
0.10
0.09
0.08
0.07
2
4
6
8
10
Epoch



FINAL RESULTS
Visible Units: 784
Hidden Units: 64
Training Epochs: 10
Final Average Free Energy: -54.576
Final Reconstruction MSE: 0.0632

5. RESULT

A Restricted Boltzmann Machine (RBM) was implemented using NumPy to learn hidden patterns from binary MNIST handwritten digit images. The energy function, hidden activation probabilities, and visible reconstruction probabilities were calculated. The RBM was trained using one-step Contrastive Divergence (CD-1) with 784 visible units and 64 hidden units. Average free energy and reconstruction error were plotted across training epochs. Original and reconstructed handwritten digit images were displayed side by side to evaluate reconstruction performance. Thus, the experiment demonstrates how RBMs learn hidden representations and reconstruct input data using an energy-based probabilistic model.